

CONTROL FLOW ABSTRACTIONS

Rank-Polymorphism, Convolution, Recursion Schemes

Laszlo Korte

<https://github.com/laszlokorte/elixir-meetup-2026-03>

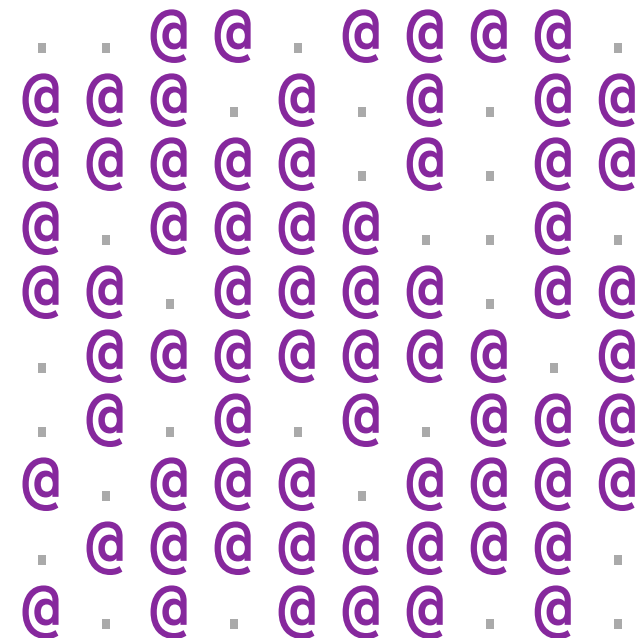
CONTENTS

1. Motivation: Advent of Code - Day 4
2. Abstractions
3. Applications
4. More Livebooks

MOTIVATION: ADVENT OF CODE - DAY 4

The **forklifts** can only access a roll of paper if there are **fewer than four rolls of paper in the eight adjacent positions**.¹

How many rolls of paper **in total** can be removed by the Elves and their forklifts?



¹<https://adventofcode.com/2025/day/4>

Multiple kinds of iteration:

Counting

- **for each** roll of paper: count the neighbors
 - **for each** neighbor: check if its a roll

Repetition

- **after each** each removal of a roll, count again

**AOC Day 4
Elixir Solution**

 [Run in Livebook](#)

ABSTRACTIONS

Applying operations to each element of an array:

```
doubled = for a <- [1,2,3] do  
  a * 2  
end
```

```
product = for {a, b} <- Enum.zip([1,2,3], [9,8,7]) do  
  a * b  
end
```

--- VS ---

```
doubled = Nx.tensor([1,2,3]) |> Nx.multiply(2)  
product = Nx.tensor([1,2,3]) |> Nx.multiply([9,8,7])
```

**Introduction to
Elixir Nx**



Run in Livebook

Combining elements with their neighborhood inside a structure (array):

```
grid = [[0,1,1,0,1, ...], [1,0,1,0,1, ...], ...]
for rows123 <- grid |> Enum.chunk_every(3, 1) do
  for row <- rows123 do
    row
    |> Enum.chunk_every(3, 1)
    |> Enum.map(&Enum.sum/1)
  end
  |> Enum.zip_with(&Enum.sum/1)
end

# --- VS ---
summed_neighbors = grid
|> Nx.tensor()
|> Nx.conv(Nx.broadcast(1, {3, 3}))
```

**Introduction to
Elixir Nx**

 Run in Livebook

[Interactive
Web Demo](#)

Producing, consuming and transforming recursive data structures:

```
Stream.unfold(5, fn 0 -> nil; n -> {n, n - 1} end)
```

```
#=> [5, 4, 3, 2, 1]
```

```
defmodule Tree do
```

```
  def rec( :empty, _fun), do: :empty
```

```
  def rec({:leaf, val}, _fun), do: {:leaf, val}
```

```
  def rec({:inner, v, l, r}, fun), do: {
```

```
    :inner, v, fun.(l), fun.(r)
```

```
}
```

```
  def grow(0), do: :empty
```

```
  def grow(1), do: {:leaf, 1}
```

```
  def grow(h), do: {:inner, h, h - 1, h - 2}
```

```
end
```

```
# generate barely balanced binary tree
```

```
Anamorphism.ana(3, &Tree.rec/2, &Tree.grow/1)
```

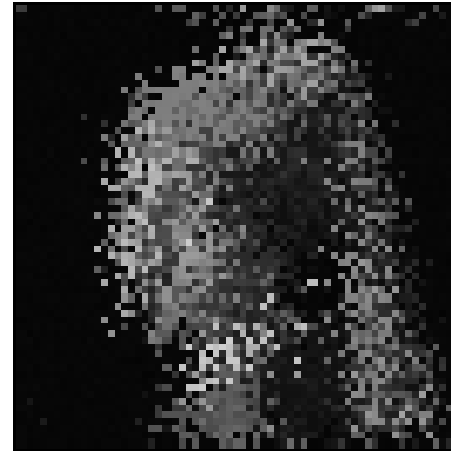
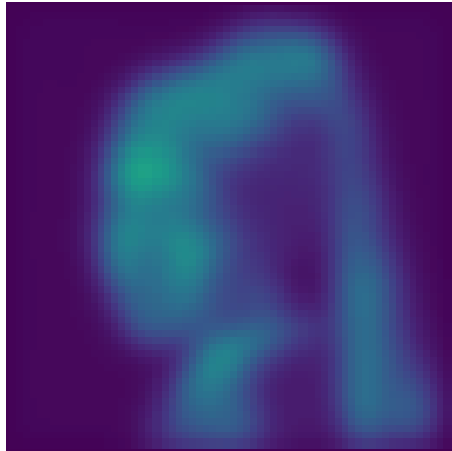
Introduction to Recursion Schemes

[Run in Livebook](#)

[Interactive](#)
[Web Demo](#)

APPLICATIONS

Generating Blue Noise for visual effects:

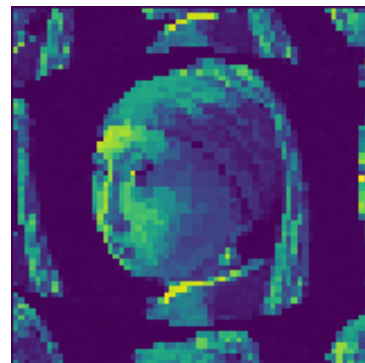
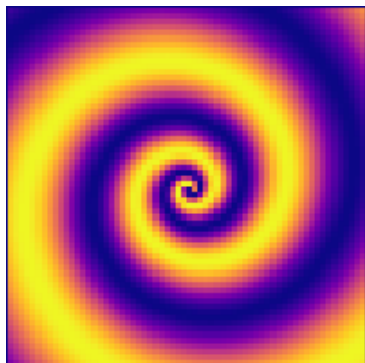
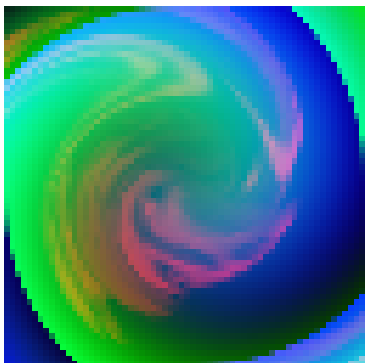
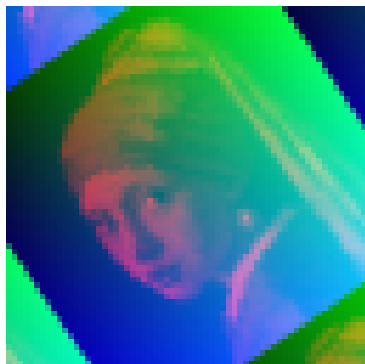


Generating Blue Noise

 [Run in Livebook](#)

[Interactive Web Demo](#)

Other image transformations:



**Introduction to
Elixir Nx**

 [Run in Livebook](#)

MORE LIVEBOOKS

- **vega_lite** can be used to render images in Livebook, **but not** Nx.Tensor
- **kino_rewind** for easy rendering of Nx.Tensor via vega_lite 
- **kino_zoetrope** for rendering 4D Nx.Tensor as slideshow **without** vega_lite 

vega_lite Example

 Run in Livebook

kino_rewind Example

 Run in Livebook

kino_zoetrope Example

 Run in Livebook

Vibe-coded experimental Elixir macro for bringing `np.einsum/torch.einsum` notation to Elixir Nx.

$$\text{Out} = \sum_i^I \sum_j^J A_{ij} B_{ji} C_j$$

```
defmodule Summation do
  import NxEinsum
  # shorthand notation for n-D sum/product
  defeinsum(weighted_trace, "ij,ji,j->")
end
```

```
out = Summation.weighted_trace(
  Nx.tensor([[3, 2], [4, 5]]),
  Nx.tensor([[1, -1], [1, 3]]),
  Nx.tensor([1, 2])
)
```

Nx.Einsum
Proof of concept

 Run in Livebook

[Interactive](#)
[Web Demo](#)

iex(16)> #  Thank you for attention

BREAK: (q) Questions

(f) Feedback (h) Go Hacking